Abstractions, Dynamic Power, and Amdahl's Law

EEL 5764 — Computer Architecture · Part A: Written Conceptual Problems (75 Points)

Problem 1 — Latency vs. Throughput Dimensions [25 Points]

Throughout, performance is defined as the reciprocal of execution time, and the CPU performance equation is used in its standard form:

```
CPU_Time = Instruction_Count  x  CPI  x  T_c        where  T_c = 1 / f
```

Part A [8 pts] — Doubling clock frequency (2.0 GHz → 4.0 GHz), IC and CPI unchanged

(1) Execution latency of the single program. Instruction count and dynamic execution behavior are held fixed by assumption, so `IC` and `CPI` are constants. Doubling `f` halves the clock period `T_c`, and execution time is directly proportional to `T_c`:

```
CPU_Time_new / CPU_Time_old = T_c_new / T_c_old = (1/4.0 GHz) / (1/2.0 GHz) = 0.5
```

> **Latency is halved** — the program's response time drops to 50% of its original value, a speedup of exactly 2.0x.

(2) Instruction throughput of the processor. Throughput is instructions retired per unit time:

```
Throughput = IPC x f = f / CPI
```

> With CPI fixed, **instruction throughput also doubles**. For a single in-order core running one unchanged single-threaded program, latency and throughput are two views of the same quantity — improving `T_c` improves both by the identical factor.

Systems-level caveat worth stating. The problem's assumption that "execution behavior remains completely unchanged" is what makes the clean 2.0x hold. On real silicon, main-memory latency is fixed in *nanoseconds*, not in cycles. A DRAM access of ~60 ns costs 120 cycles at 2.0 GHz but 240 cycles at 4.0 GHz, so `CPI` inflates and the measured speedup falls below 2.0x. This is the Memory Wall: doubling the clock does not double delivered performance once off-chip accesses dominate.

Part B [8 pts] — Adding 4 identical independent cores, clock frequency unchanged

(1) Response time of a single-threaded database lookup. A single thread executes on exactly one core at a time. The added cores do not reduce `IC`, do not reduce that thread's `CPI`, and do not change `T_c`:

> **Single-thread latency is essentially unchanged.** In practice it may degrade slightly: additional cores contend for the shared last-level cache, on-die interconnect, and memory controllers, and the larger power/thermal footprint often forces a lower single-core turbo frequency. Extra cores never accelerate a serial dependency chain.

(2) Transaction throughput of a highly parallel web-server workload. Web-server requests are independent tasks, so they map onto cores as concurrent work:

```
Throughput_new / Throughput_old  ->  N_new / N_old   (ideal, perfectly parallel)
```

> **Aggregate throughput scales up roughly with core count** (e.g., ~5x for a 1-core to 5-core change under ideal scaling). Realized scaling is sub-linear, bounded by Amdahl's Law on the serial fraction of request handling, plus shared-resource limits: memory bandwidth and LLC capacity per core, lock contention on shared data structures, OS scheduling overhead, and I/O saturation.

The architectural moral. Spatial scaling (more cores) buys throughput, not latency. This is precisely the post-Dennard multicore pivot: with frequency capped by the Power Wall, the industry purchased *throughput* with parallelism because *latency* could no longer be purchased with clock speed.

Part C [9 pts] — How the target workload dictates the optimization objective

The metric an architect optimizes is not a matter of taste; it is dictated by what the workload's consumer actually perceives. The decision rule is:

- **Optimize for latency** when a *human being or a physical process is blocked waiting* on the completion of one task, and when tasks arrive one at a time so there is no concurrency to exploit. The relevant unit is seconds-per-task at the tail of the distribution. Architectural levers: deep speculation, aggressive branch prediction, wide out-of-order issue, large low-latency private caches, high clock frequency.
- **Optimize for throughput** when there is a *large queue of independent tasks* and the cost function is work completed per second per watt per dollar. Any single task may be slow so long as aggregate rate is high. Architectural levers: many simple cores, SIMD/vector width, hardware multithreading (SMT) to hide stalls, HBM/high memory bandwidth, domain-specific accelerators.

Objective	Real-world example	Why that metric dominates
Response time (latency-optimized)	An automotive airbag / anti-lock braking ECU (an embedded real-time controller)	A single sensor-to-actuator decision must complete within a hard millisecond-scale deadline. There is exactly one task in flight and no aggregate rate to amortize against; a missed deadline is a safety failure, so worst-case single-task latency is the only metric that matters. (A high-frequency trading engine is the same argument in a commercial setting: the profit is captured by whoever responds first.)
Bandwidth (throughput-optimized)	A warehouse-scale AI training cluster (e.g., GPU/TPU racks training a large model)	The job is a fixed enormous volume of independent matrix operations. No user waits on any individual kernel; the objective is total FLOPs delivered per second within a megawatt power budget, so the design maximizes parallel execution units and HBM bandwidth and accepts high per-operation latency, hiding it with massive thread-level parallelism.

The hybrid reality. Most modern systems are constrained on both axes simultaneously. A datacenter serving interactive search must maximize queries per second (throughput, which sets cost per query) while holding a tail-latency service-level objective such as "99th percentile under 100 ms" (latency, which sets user satisfaction). Heterogeneous SoCs resolve this by building both: large performance cores for latency-critical threads and clusters of efficiency cores for parallel background throughput, within one shared power envelope.

Problem 2 — CMOS Dynamic Power Scaling & The Power Wall [25 Points]

Governing equation and baseline. Dynamic switching power in CMOS is:

```
P_dynamic = C x V^2 x f x alpha
```

with the 14 nm baseline given as `V_0 = 1.20 V`, `f_0 = 3.5 GHz`, `P_0 = 25 W`, capacitance `C_0`, and activity factor `alpha` held constant. Because `alpha` is constant and `C_0` is never given numerically, the correct method is a **ratio derivation**: form `P_new / P_0` so that `C_0` and `alpha` cancel identically.

```
P_new / P_0 = (C_new / C_0) x (V_new / V_0)^2 x (f_new / f_0)
```

Part A [10 pts] — 7 nm shrink: C reduced 30%, V = 0.95 V, f = 4.0 GHz

Step 1 — Capacitance factor. A 30% reduction leaves 70% of the original capacitance:

```
C_new = 0.70 x C_0        =>  C_new / C_0 = 0.7000
```

Step 2 — Voltage factor (squared, and therefore the dominant term):

```
(V_new / V_0)^2 = (0.95 / 1.20)^2 = (0.791667)^2 = 0.9025 / 1.44 = 0.626736
```

Step 3 — Frequency factor (linear):

```
f_new / f_0 = 4.0 / 3.5 = 1.142857
```

Step 4 — Combine the three independent ratios:

```
P_new / P_0 = 0.7000 x 0.626736 x 1.142857
            = 0.438715 x 1.142857
            = 0.501389
```

Step 5 — Scale by the baseline power:

```
P_new = 0.501389 x 25 W = 12.5347 W
```

> **P_new ≈ 12.53 W** — approximately 50.1% of the baseline, i.e. the core consumes *half* the dynamic power while running *14% faster*. Note where the win comes from: the capacitance shrink contributed a factor of 0.70, but the modest 0.25 V voltage drop contributed 0.627 on its own, and that squared term more than paid for the entire frequency increase.

Part B [10 pts] — Frequency boost to 4.8 GHz requiring V = 1.15 V (same 7 nm node)

The node is unchanged, so the capacitance factor remains 0.70. Only `V` and `f` change:

```
(V_boost / V_0)^2 = (1.15 / 1.20)^2 = 1.3225 / 1.44 = 0.918403
f_boost / f_0     = 4.8 / 3.5 = 1.371429

P_boost / P_0 = 0.7000 x 0.918403 x 1.371429
              = 0.642882 x 1.371429
              = 0.881667

P_boost = 0.881667 x 25 W = 22.0417 W
```

> **P_boost ≈ 22.04 W** — 88.2% of the original 14 nm power budget. Nearly the entire energy-efficiency benefit of the process shrink has been surrendered to buy the last 0.8 GHz.

Part C [5 pts] — Comparison, the voltage floor, and the physical origin of the Power Wall

Configuration	V (V)	f (GHz)	P (W)	vs. 14 nm baseline	W per GHz
Baseline (14 nm)	1.20	3.5	25.00	—	7.14
Part A (7 nm, scaled)	0.95	4.0	12.53	−49.9%	3.13
Part B (7 nm, boosted)	1.15	4.8	22.04	−11.8%	4.59

The comparison, quantified. Moving from Part A to Part B buys 20% more clock frequency (4.0 → 4.8 GHz) at a cost of 75.8% more power (12.53 → 22.04 W). Decomposing exactly which term is responsible:

```
P_boost / P_new = (f_boost / f_new) x (V_boost / V_new)^2
                = 1.200        x  (1.15 / 0.95)^2
                = 1.200        x  1.4654
                = 1.7584   (a 75.8% power increase)
```

> Only a factor of **1.20** came from the frequency itself. The remaining factor of **1.47** — the larger share — came purely from the voltage being forced back up. Frequency scaling is linear in power; the voltage it demands is quadratic.

Why the voltage floor exists. To switch a transistor faster, gate capacitance must be charged and discharged more quickly, which requires higher gate overdrive (`V_supply − V_threshold`). Supply voltage can only be lowered by also lowering the threshold voltage `V_th` — but subthreshold leakage current grows exponentially as `V_th` falls, so static power `P_static = I_leakage x V_supply` explodes. Below roughly 1.0 V in sub-micron silicon, further reduction also collides with gate-oxide tunneling, process variability (transistors on the same die no longer switch identically), and noise-margin/reliability limits. Voltage therefore hits a hard floor.

How this is the physical origin of the Power Wall. Dennard scaling worked precisely because `V` scaled down by `1/K` alongside dimensions: the `V^2` term absorbed the frequency increase automatically, so power density stayed flat and performance rose "for free." Once `V` stopped scaling around 2005, that automatic compensation vanished. The arithmetic above is exactly what remains: with `V` pinned near its floor, every additional GHz must be paid for linearly in `f` *and* quadratically in whatever voltage increase that frequency requires for stable timing closure. Power density rises faster than any economically viable cooling solution can remove heat.

The consequences are the defining features of modern architecture: clock frequencies have been effectively capped near 3–5 GHz for two decades; **dark silicon** emerges because transistor density keeps growing faster than the power budget, so not all of the die can switch at full speed simultaneously; and the industry pivoted from frequency scaling to multicore parallelism and domain-specific accelerators, which deliver performance per watt rather than performance per hertz.

Problem 3 — Speedup Limits and Amdahl's Law [25 Points]

Given: fraction enhanced `f = 0.65` (matrix operations), local speedup of the NPU `s = 40`.

```
Speedup_overall = 1 / ( (1 - f) + (f / s) )
```

Part A [10 pts] — Overall system speedup from integrating the NPU

Step 1 — Identify the unenhanced fraction. This is the portion of baseline execution time the NPU cannot touch (data movement, control flow, activations, normalization, I/O, OS overhead):

```
1 - f = 1 - 0.65 = 0.35
```

Step 2 — Compute the new duration of the enhanced fraction. The 0.65 of execution time that runs 40x faster now occupies:

```
f / s = 0.65 / 40 = 0.01625
```

Step 3 — Sum to obtain the new normalized execution time. Taking baseline execution time as 1.0:

```
T_new = 0.35 + 0.01625 = 0.36625
```

Step 4 — Invert to obtain speedup:

```
Speedup_overall = 1 / 0.36625 = 2.730375...
```

> **Speedup_overall ≈ 2.73x** (execution time falls to 36.6% of baseline).

Read the structure of that number. A 40x hardware accelerator delivered only 2.73x at the system level. In the accelerated system the enhanced work is now just 0.01625 / 0.36625 = 4.4% of total runtime, while the untouched 35% has grown to **95.6% of all execution time**. The NPU did its job so thoroughly that it eliminated itself as the bottleneck.

Part B [8 pts] — Theoretical ceiling as s → infinity

```
Speedup_limit = lim (s -> inf)  1 / ( (1 - f) + (f / s) )

As s -> infinity, the term  f / s = 0.65 / s -> 0,
so the expression collapses to:

Speedup_limit = 1 / (1 - f) = 1 / 0.35 = 2.857142...
```

> **Maximum theoretical speedup ≈ 2.86x**, even if the matrix operations executed in literally zero time.

The engineering punchline. The already-built 40x NPU achieves 2.73x, which is **95.6% of the 2.86x ceiling**. Every remaining month of hardware effort — pushing the local speedup from 40x all the way to infinity — is competing for a residual 4.4% of system performance. The correct engineering decision is to stop optimizing the NPU and redirect the team at the 35% serial remainder.

Local NPU speedup s	Enhanced portion (f/s)	Total time	Overall speedup	% of ceiling
1 (baseline)	0.6500	1.00000	1.00x	35.0%
10	0.0650	0.41500	2.41x	84.3%
40 (as built)	0.0163	0.36625	2.73x	95.6%
100	0.0065	0.35650	2.80x	98.2%
∞	0.0000	0.35000	2.86x	100.0%

Part C [7 pts] — The bottleneck, and why architects "make the common case fast"

The bottleneck: the sequential (unenhanced) fraction. The 35% of baseline execution time that the NPU cannot accelerate is an absolute floor on execution time. It does not shrink, it does not overlap, and no amount of acceleration applied elsewhere reduces it. In this ML workload it is concretely composed of:

- **Data movement.** Staging tensors between DRAM, the CPU's cache hierarchy, and NPU local scratchpad memory. This is bandwidth- and latency-bound, not compute-bound, so a faster matrix engine does not help — it is the Memory Wall reasserting itself inside the accelerated system.
- **Serial control flow.** Python/framework graph dispatch, kernel launch overhead, synchronization barriers between layers, and branch-heavy scheduling logic that must run on the general-purpose CPU.
- **Non-matrix operators.** Activation functions, normalization layers, pooling, loss computation, and pre/post-processing that fall outside the NPU's specialized datapath.
- **System-level overhead.** OS scheduling, driver calls, file and network I/O for loading batches.

Why "make the common case fast" is the governing principle. Amdahl's Law makes the priority ordering mathematical rather than intuitive. Overall gain is bounded by `1/(1-f)`, so *f dominates s*: the fraction of time a mechanism affects is a harder constraint than how much it accelerates that fraction. Contrast two hypothetical design allocations:

```
Rare case:   f = 0.01, s = 100  ->  Speedup = 1 / (0.99 + 0.0001) = 1.01x
Common case: f = 0.65, s = 2    ->  Speedup = 1 / (0.35 + 0.325) = 1.48x
```

A 100x accelerator aimed at 1% of runtime is worth 1% overall. A merely 2x improvement aimed at 65% of runtime is worth 48% overall — nearly fifty times more valuable, for a fraction of the design cost. Since silicon area, power budget, verification effort, and engineer-hours are all strictly finite, spending them on a complex, rarely-exercised sub-module is quantifiably poor resource allocation, and it also carries the dark-silicon penalty: that block still consumes leakage power and area while sitting idle.

The correct next move for this design team. Amdahl's Law is not only a limit — it is a profiler-driven roadmap. Having accelerated the 65%, the new common case *is* the 35%. The team should re-profile, decompose that remainder, and attack its largest components: overlap data movement with compute via double-buffering and DMA prefetch, fuse activation and normalization kernels into the NPU pipeline to convert unenhanced work into enhanced work, and reduce launch and synchronization overhead. Note that raising `f` from 0.65 to 0.85 lifts the ceiling from 2.86x to 6.67x — a far larger prize than anything left in `s`.

All numerical results verified independently by computation. Values rounded to two decimal places as instructed; intermediate ratios carried at six significant figures.